

Decision Doc: Automated RFQ to Quote Drafting

Problem 1 · Fastlane AI Agent Engineer Assessment

What I built and why

I chose the RFQ problem: turning an incoming distributor RFQ into a drafted quote and reply email. I picked it over the other one, the marketing content multiplier. That one asks for two kinds of visual work, a brand-locked design and a generated image, plus a consistent voice across formats, so in a timed build my hours would have gone into design rather than the agent I am actually being judged on. The RFQ problem is more contained, and it comes down to matching parts, pricing them right, and knowing when to stop and ask a person, which is a call I can stand behind when someone pushes on it. It is close to work I have done before in email automation and inventory, so my time went into the hard parts instead of catching up.

The hard part here is not writing text, it is judgment. The system has to read a plain sentence like “hydraulic hose, 2 wire braid, 3/4 inch,” match it to the exact catalog part, and know when it should not guess. That one idea shaped everything: the AI handles language, ordinary code handles anything involving money, and a person approves the quote before it goes out.

What I built and what I chose not to build

I built the workflow as a small, ordered pipeline. The AI does only two jobs: it reads the RFQ into a clean list of line items, and it writes the reply email at the end. Everything in between, matching parts, looking up prices, and adding up totals, is plain code I can test, because anything that touches money has to be predictable. I deliberately kept that logic in code rather than hiding it inside a drag-and-drop automation tool, where it would be harder to test and explain.

The most important decision was not letting the AI pick the catalog part itself. It sounds confident even when it is wrong, and the reason it gives is written after it has already chosen, so it is a justification, not real logic. A confidently wrong part number is the worst thing that can happen here. So the rule is simple: if a match is not certain, it goes to a person instead of being guessed. A slower answer is fine; a wrong one is not.

How I use the AI, and how I keep it in bounds

At each of the two AI steps the model gets a narrow job and firm rules; the exact instructions are included separately. When reading the RFQ, it is told it transcribes, it does not decide. It answers in a strict format we validate automatically, keeps the customer’s exact wording for every line, records a part number only when one actually appears and never invents one, and gives a confidence score it is told never to inflate. A few rules cover the traps we saw in real requests: telling a quantity apart from a fixed length, handling wording like “steel, not stainless,” keeping a useful word like “pneumatic” while setting aside a vague one like “chemical service” instead of guessing a product from it, always capturing fitting ends, and sorting out loose dates.

When writing the reply email, the AI only writes the wording around numbers the code already worked out. The single dollar figure it is allowed to state is the subtotal, exactly as given, and the code checks that afterward and rewrites the email if it drifts. It also has to name every flagged line and ask the specific question a salesperson would. In short, the AI never touches a price or a match.

How the system works

The system works in seven steps, with a stop in the middle where a person reviews the quote before anything goes out.

- 1. Read the file.** The RFQ comes in as a PDF or an email and is turned into plain text.
- 2. Extract the details.** The AI pulls out each line item, quantity, and product detail, with a confidence score and the exact original wording kept for review.
- 3. Match the catalog.** An exact part number is trusted right away. A part number that is not in the catalog is flagged, never guessed at. A line with no part number goes through a scoring step that checks size, pressure, material, and fittings, and anything under spec, such as a hose rated for less pressure than requested, is rejected rather than treated as close enough.
- 4. Add pricing.** Unit price, lead time, and totals come straight from the catalog using exact arithmetic with no rounding errors, never from the AI. If an item is short on stock or cannot make the requested date, the quote flags

it.

5. Put the quote together. Each line shows what the customer asked for, what was matched and why, and the price, and the system drafts the reply email with a PDF quotation attached.

6. Review. The process pauses for a person and will not continue while any item is unresolved.

7. Finalize. Once approved, the system recalculates every number from the final edited values, sends the reply email, and writes a record for the accounting system. Both the send and the accounting write are simulated in this prototype.

What happens when something goes wrong

If the AI returns something that does not fit the expected format, the system rejects it, tries once more with the error pointed out, and then stops with a clear message rather than passing bad data forward. If it returns something that looks reasonable but is missing a detail, the matching step catches it: in testing, one extraction dropped the word “pneumatic” from a line, and the result was a line flagged for review and left out of the total, not a wrong match. Low-confidence and tied matches always go to a person as a question, never a guess.

If a reviewer changes something, such as a quantity, every number is recalculated from the edited values, so an edited line can never keep an old price by mistake. And if the accounting system were ever unreachable when a quote is approved, the quote would stay approved but not yet recorded, with the reply email held until the record goes through, since an unrecorded quote is a bigger problem than a slightly late reply.

The review step is real, not a formality. The reviewer sees the reasoning behind every line, can change any field, and must resolve every flagged item before approval. On a flagged line they can accept the suggested match, choose or type a different part, mark it as not currently available, or remove it. One unavailable item does not block the rest of the quote, and the reviewer can still reject the whole quote.

How I verified it

Because the whole point is trust, I focused testing on the parts that are never allowed to be wrong, and kept most of those tests away from the AI so they run instantly and for free. Without any AI calls, I check that each request is complete and correctly formed, that the matching rules behave (an exact part number is trusted, an under-spec part is rejected, a tie is flagged), and that every price and total is exact. With the AI, I run extraction on the real sample files, the subtotal check on the email, and a full run-through three ways: a clean approval with an edit, a flagged line resolved by a reviewer, and a rejection. One command runs all of it and ends with a single pass line.

How it would connect to real business systems

Today the system writes the accounting record to a file instead of sending it anywhere. In a real version the reviewer approves first, and nothing goes to an outside system before that. The system would then sign in to the accounting software (Sage Intacct) with securely stored credentials, check whether the customer already exists and create them if not, then create the quote with one line per approved item and a link back to the original RFQ. It is built so the same quote can never be recorded twice. If a write fails it retries, and after repeated failures the quote is marked approved but not recorded, someone is notified, and the email is held until the record is confirmed. A real email send works the same way, behind the same approval step, and is off by default.

What would carry over to other clients

The reusable part is the overall shape: read a document, use AI to pull out structured details, run them through a scoring step that only accepts confident matches, build a priced quote with a reason for every line, pause for a person, recalculate on any edit, then send. What changes for each new client is smaller: their catalog and how it is scored, the words their customers use, and how the final record maps into their systems. To confirm it holds up beyond the four samples, I wrote a fresh RFQ myself in a format the system had never seen, and it handled it correctly from start to finish, including flagging the items it should not guess.

What is left to do next

With more time I would connect a live inbox so new RFQs start automatically, build the real accounting connection behind the same approval step, and let the matching improve from the corrections reviewers make over time. I would finish support for scanned RFQs, which is partly built but not yet tested. As the catalog grows much larger, the matching step would need a faster way to shortlist candidates before scoring them, without changing how the scoring works. And I would grow the current checks into a larger test set built from real past RFQs and their approved quotes.